

Spring Framework Notes

Day 38 — SpringJDBC Day 06 : Product Tracking Project

Project Requirement

Project Requirement — Product Tracking

Fields to be tracked (stored in the database):

- ProductName — DB
- ProductShippingAddress
- ProductPrice
- Quantity
- PlacedDate

Operations to be implemented, and who is responsible for each:

- Price of the Product based on name — [Student]
- Get all the products — [Student]
- Insert the product — [Prashant]
- Delete the product — [Student]
- Update the product — [Prashant]
- Find and count all the products based on name — [Prashant]
- Place all the products based on address — [Student]

Project Structure

- ProductTrackingDao — [@Repository]
 - JdbcTemplate — [@Autowired]
- ProductBo — [Lombok API's]
- SpringJDBC — [Java Configuration] [@Configuration] [@ComponentScan]
 - JdbcTemplate, DataSource
 - » DriverManagerDataSource
 - Environment — [@Autowired]
 - application.properties — [@PropertySource()]
- Main user
 - AnnotationConfigApplicationContext

Entity Class

ProductDelivery.java

```
package com.projectBackup;

import java.util.Date;

@Data
public class ProductDelivery {
    private String productName;
    private String productShippingAddress;
    private float productPrice;
    private int productQuantity;
    private Date productPlaceDate;
}
```

Repository Class — OperationDao

This class is annotated with `@Repository` and uses `JdbcTemplate`, injected through constructor injection with `@Autowired`, to perform all the database operations for the Product Tracking project.

OperationDao.java

```
package com.projectBackup;

import java.time.LocalDate;
import java.util.Date;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

@Repository("operation")
public class OperationDao {

    private JdbcTemplate template;

    private static final String SQL_QUERY_FOR_GETPRODUCTPRICE=
        "select price from ProductDetail where productName=?";

    private static final String SQL_QUERY_FOR_GETALLPRODUCT=
        "select * from ProductDetail";

    private static final String SQL_QUERY_FOR_INSERTPRODUCT=
        "insert into ProductDetail
(productName,shippingAdd,price,quantity,placeDate) values(?,?,?, ?,?)";
```

```

private static final String SQL_QUERY_FOR_DELETEPRODUCT=
    "delete from ProductDetail where productName=?";

private static final String SQL_QUERY_FOR_UPDATEPRODUCT=
    "update ProductDetail set shippingAdd=? , placeDate=? where
productName=?";

@Autowired
public OperationDao(JdbcTemplate template) {
    super();
    this.template = template;
}

public Float getProductPrice(String name) {
    return template.queryForObject
        (SQL_QUERY_FOR_GETPRODUCTPRICE, Float.class,new Object[]
{name});
}

public List<ProductDelivery> getAllProduct(){
    return template.query(SQL_QUERY_FOR_GETALLPRODUCT,
        (rs,rowNo)->{
            ProductDelivery delivery=new
ProductDelivery();

            delivery.setProductName(rs.getString(1))
;

            delivery.setProductShippingAddress(rs.ge
tString(2));

            delivery.setProductPrice(rs.getFloat(3))
;

            delivery.setProductQuantity(rs.getInt(4)
);

            delivery.setProductPlaceDate(rs.getDate(
5));

            return delivery;
        });
}

public int insertProduct(ProductDelivery delivery) {
    return template.update(SQL_QUERY_FOR_INSERTPRODUCT,
        delivery.getProductName(),
        delivery.getProductShippingAddress
(),

        delivery.getProductPrice(),
        delivery.getProductQuantity(),
        delivery.getProductPlaceDate());
}

public int deleteProduct(String name) {
    return template.update(SQL_QUERY_FOR_DELETEPRODUCT,
        name);
}

public int updateProduct(String address,LocalDate date,String name) {
    return template.update(SQL_QUERY_FOR_UPDATEPRODUCT,address,date,name);
}

```

```
}  
}
```

Java Configuration — OperationConfig

This class provides the `DataSource` and `JdbcTemplate` beans required by SpringJDBC, reading the database connection details from `application.properties` through the `Environment` object, and enables component scanning for the project package.

OperationConfig.java

```
package com.projectBackup;  
  
import javax.sql.DataSource;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.context.annotation.Bean;  
import org.springframework.context.annotation.ComponentScan;  
import org.springframework.context.annotation.Configuration;  
import org.springframework.context.annotation.PropertySource;  
import org.springframework.core.env.Environment;  
import org.springframework.jdbc.core.JdbcTemplate;  
import org.springframework.jdbc.datasource.DriverManagerDataSource;  
  
@Configuration  
@ComponentScan("com.projectBackup")  
@PropertySource("classpath:com/projectBackup/application.properties")  
public class OperationConfig {  
  
    @Autowired  
    Environment environment;  
  
    @Bean  
    public DataSource dataSource() {  
        DriverManagerDataSource source=new DriverManagerDataSource();  
  
        source.setDriverClassName(environment.getProperty("db.driverName"));  
        source.setUrl(environment.getProperty("db.url"));  
        source.setUsername(environment.getProperty("db.username"));  
        source.setPassword(environment.getProperty("db.password"));  
  
        return source;  
    }  
  
    @Bean  
    public JdbcTemplate jdbcTemplate() {  
        JdbcTemplate template=new JdbcTemplate(dataSource());  
        return template;  
    }  
}
```

```
}
```

Main Class — Console Application

The Main class starts the IoC container using `AnnotationConfigApplicationContext`, fetches the `OperationDao` bean by its bean name, and drives a menu-based console application to perform all the product tracking operations.

Main.java

```
package com.projectBackup;

import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.util.List;
import java.util.Scanner;
import java.util.stream.Collectors;
import java.util.stream.Collectors;

import org.springframework.context.annotation.AnnotationConfigApplicationContext;

public class Main {
    @SuppressWarnings("unlikely-arg-type")
    public static void main(String[] args) {
        System.out.println("Container started");
        AnnotationConfigApplicationContext context = new
        AnnotationConfigApplicationContext(OperationConfig.class);

        OperationDao bean = context.getBean("operation", OperationDao.class);
        Scanner sc = new Scanner(System.in);
        while (true) {
            System.out.println("\u001B[31m" + "Product Tracking");
            System.out.println("\u001B[32m" + "Press 1 for getting the Product
Price");
            System.out.println("\u001B[33m" + "Press 2 for getting all the
Product Details");
            System.out.println("\u001B[34m" + "Press 3 for Adding the new
Product");
            System.out.println("\u001B[35m" + "Press 4 for deleting the
Product");
            System.out.println("\u001B[36m" + "Press 5 for updating the
Product");
            System.out.println("\u001B[32m" + "Press 6 for finding and count the
Product by it's name");
            System.out.println("\u001B[33m" + "Press 7 for grouping all the
Product based on shipping address");
            System.out.println("Press 8 for exits" + "\u001B[0m");
            System.out.println("");
            System.out.println("Enter your choice");
            int choice = sc.nextInt();

            switch (choice) {
                case 1:
```

```

        try {
            System.out.println("Enter the product name");
            String name = sc.next();
            Float productPrice = bean.getProductPrice(name);
            System.out.println("Price of the Product" + name + " is Rs."
+ productPrice);
        } catch (org.springframework.dao.EmptyResultDataAccessException
e) {
            System.out.println("No Product found with this name");
        }
    /*
    * if(productPrice!=null) {
    * System.out.println("Price of the Product"+name+" is
Rs."+productPrice); }
    * else { System.out.println("No Product found with this name");
    }
    */
    break;
case 2:
    List<ProductDelivery> allProduct = bean.getAllProduct();
    if (!allProduct.isEmpty()) {
        allProduct.stream().forEach(System.out::println);
    } else {
        System.err.println("Cart is empty!");
    }

    break;
case 3:
    System.out.println("Enter the Product name");
    String name = sc.next();
    System.out.println("Enter the shipping address");
    String address = sc.next();
    System.out.println("Enter the price");
    float price = sc.nextFloat();
    System.out.println("Enter the product quantity");
    int quantity = sc.nextInt();
    System.out.println("Enter the date of placing the order");
    String input = sc.next();

    DateTimeFormatter format =
DateTimeFormatter.ofPattern("dd-MM-yyyy");
    LocalDate date = LocalDate.parse(input, format);
    java.sql.Date sqlDate = java.sql.Date.valueOf(date);

    ProductDelivery data = new ProductDelivery();
    data.setProductName(name);
    data.setProductShippingAddress(address);
    data.setProductPrice(price);
    data.setProductQuantity(quantity);
    data.setProductPlaceDate(sqlDate);

    int result = bean.insertProduct(data);

    if (result >= 1) {
        System.out.println("Data inserted");
    } else {

```

```

        System.out.println("Data not inserted");
    }

    break;
case 4:
    System.out.println("Enter the product name for delete");
    String datafordelete = sc.next();

    int resultForDelete = bean.deleteProduct(datafordelete);

    if (resultForDelete >= 1) {
        System.out.println("Data Deleted!");
    } else {
        System.out.println(" Name :" + datafordelete + " with this
name there is no any product");
    }
    break;
case 5:
    System.out.println("Enter the shipping address to update");
    String updateAddress = sc.next();
    System.out.println("Enter the date of updating");
    String updateDate = sc.next();

    DateTimeFormatter formatter =
DateTimeFormatter.ofPattern("dd-MM-yyyy");
    LocalDate newdate = LocalDate.parse(updateDate, formatter);

    java.sql.Date newValue = java.sql.Date.valueOf(newdate);

    System.out.println("Enter the name of the product for update");
    String nameOfProduct = sc.next();

    int updateResult = bean.updateProduct(updateAddress, newdate,
nameOfProduct);

    if (updateResult >= 1) {
        System.out.println("Record Updated");
    } else {
        System.out.println("Name :" + updateResult + " is not
present");
    }

    break;
case 6:
    System.out.println("Enter the name for finding the product");
    String nameFinding=sc.next();

    List<ProductDelivery> find=bean.getAllProduct();

    long count = find.stream().filter(e ->
e.getProductName().equals(nameFinding))

        .count();

    System.out.println("With name: "+nameFinding+" product are of
number :"+count);

```

```
        break;
    case 7:
        break;
    case 8:
        break;
    default:
        break;
    }
}
}
}
```

Configuration & Database

application.properties

```
db.driverName=oracle.jdbc.driver.OracleDriver
db.url=jdbc:oracle:thin:@localhost:1521:xe
db.username=system
db.password=manager
```

Database Schema

Name	Null?	Type
PRODUCTNAME		VARCHAR2 (20)
SHIPPINGADD		VARCHAR2 (20)
PRICE		NUMBER (10,2)
QUANTITY		NUMBER (10)
PLACEDATE		DATE